CGL-sheet09

Robin Kühnberger[1] and Felix Naraschewski[2]

[1]University Stuttgart
[2]University Stuttgart

December 21, 2025

Contents

1 Fitten von Funktionen 1

2 SymPy 2

3 Primzahlen 4

1 Fitten von Funktionen

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.optimize as opt

data = np.loadtxt("data.txt", dtype=float, delimiter=",")
x = data[:, 0]
y = data[:, 1]

def damped_sine(x, a, d, b):
    return a * np.exp(-d * x) * np.sin(b * x)

param, param_cov = opt.curve_fit(damped_sine, x, y)


print(f"Fit Results: Amp={param[0]:.3f}, Damping={param[1]:.3f},
↪  Freq={param[2]:.3f}")

y_fit = damped_sine(x, *param)
```

```
plt.plot(x, y, 'o', color='red', label="data",)
plt.plot(x, y_fit, '-', color='blue', label="fit")

plt.title("Damped Oscillation")
plt.legend()
plt.savefig("fitten-von-funktionen.pdf")
plt.show()
```

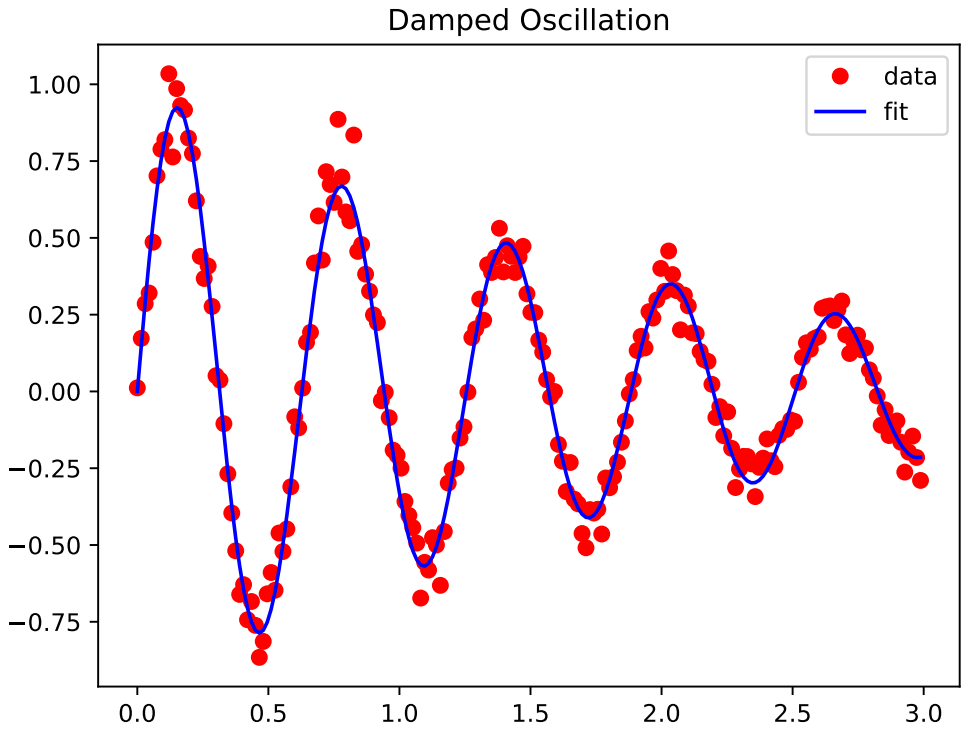


Figure 1: Fitten von Funktionen Plot

2 SymPy

```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt

t = sp.symbols('t', real=True)
```

```
A = sp.symbols('A')
lam = sp.symbols('lambda', positive=True, real=True)
omega = sp.symbols('omega', positive=True, real=True)

f = A * sp.exp(-lam * t) * sp.cos(omega * t)

f_prime = sp.diff(f, t)

f_integral = sp.integrate(f, t)

f_limit = sp.limit(f, t, sp.oo)

f_func_np = sp.lambdify((t, A, lam, omega), f, 'numpy')
df_func_np = sp.lambdify((t, A, lam, omega), f_prime, 'numpy')

A_val = 2
lam_val = 0.5
omega_val = 3

t_vals = np.linspace(0, 10, 1000)

y_f = f_func_np(t_vals, A_val, lam_val, omega_val)
y_df = df_func_np(t_vals, A_val, lam_val, omega_val)

plt.plot(t_vals, y_f, label="Schwingung", color='blue')
plt.plot(t_vals, y_df, label="Ableitung", color='red')

plt.title("Gedämpfte Schwingung und 1. Ableitung")
plt.xlabel("Zeit (t)")
plt.ylabel("Amplitude")
plt.legend()
plt.savefig("SymPy.pdf")
plt.show()
```

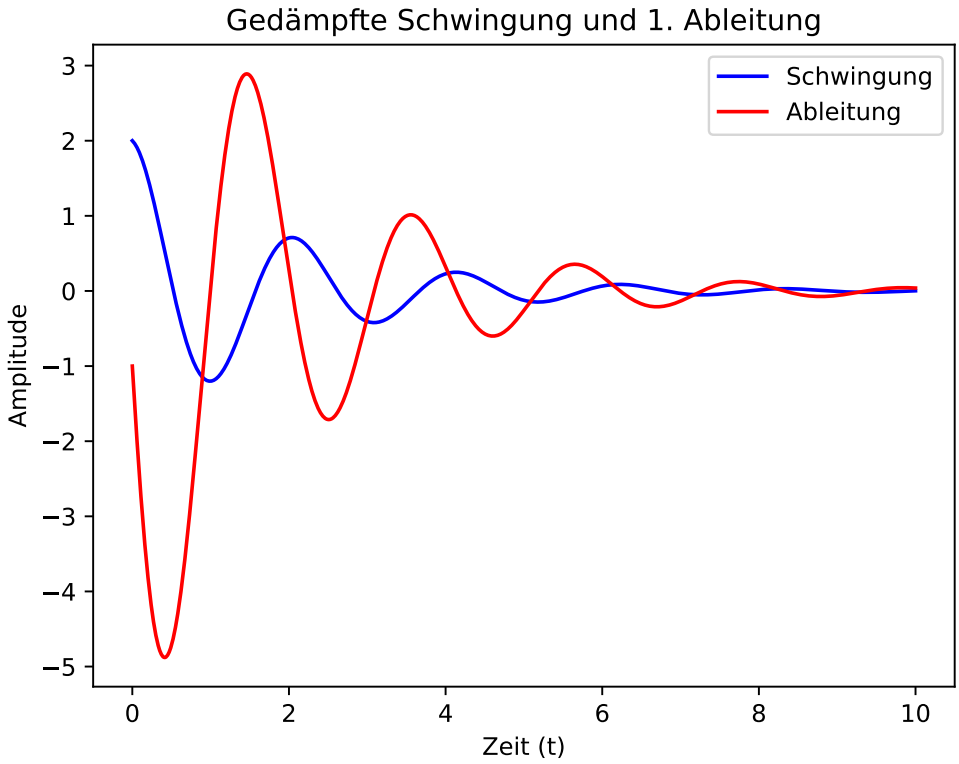


Figure 2: SymPy Plot

3 Primzahlen

```python
import numpy as np

def sieb_des_eratosthenes(n):
    if n < 2:
        return []
    ist_prim = [True] * (n + 1)
    ist_prim[0] = ist_prim[1] = False

    for p in range(2, int(np.sqrt(n)) + 1):
        if ist_prim[p]:
            for i in range(p * p, n + 1, p):
                ist_prim[i] = False

    return [i for i, prim in enumerate(ist_prim) if prim]
```

```
n_limit = input("Enter a limit (N): ")
n_limit = int(n_limit)
print(f"Primzahlen bis {n_limit}: {sieb_des_eratosthenes(n_limit)}")
```

List of Figures

1 Fitten von Funktionen Plot 2
2 SymPy Plot 4